

THE SOFTWARE ARCHITECT CODEX

From Code to Cloud: A Complete Handbook for Building World-Class Software Systems

Authors: A collective of engineers from Google, Amazon, Netflix, Meta, OpenAI, Stripe, and MIT

Version: 1.0

Difficulty: Apprentice → Architect

PREFACE: WHY THIS BOOK EXISTS

Every year, thousands of engineers graduate from bootcamps and universities knowing how to write code — but not knowing how to **build systems**.

There is a gap between writing a function and designing a platform that serves millions. This book exists to bridge that gap.

The software industry has spent decades accumulating knowledge about what works and what doesn't. The problem is that this knowledge is scattered across hundreds of blog posts, thousands of Stack Overflow answers, years of tribal knowledge inside companies, expensive conference talks, and internal wikis you'll never see.

The Software Architect Codex collects this knowledge into one complete learning journey.

We will not teach you syntax. Syntax changes every year. We will teach you **principles**. Principles last decades.

What You Will Become

By the end of this handbook, you will be able to:

- Design systems that serve millions of users
- Make informed trade-offs between competing architectural approaches
- Debug production issues under pressure
- Design databases that scale
- Deploy and operate cloud-native applications
- Build AI-powered features into any application
- Interview at the world's top technology companies
- Think like a Staff+ engineer

How This Book Works

Every chapter follows the same structure:

1. **Why this exists** — The problem it solves
2. **Core concepts** — The mental models you need
3. **Visual explanations** — Diagrams and flowcharts
4. **Implementation** — Hands-on code that you write
5. **Senior insights** — What experienced engineers know
6. **Exercises** — To cement your understanding

The Master Project: Nexus

Throughout this book, you will build **one** evolving platform. We'll call it **Nexus**.

Nexus starts as a simple authentication service. By the end, it will be a distributed platform combining features from Notion, Slack, Discord, Google Drive, GitHub, Jira, and ChatGPT.

Every chapter adds to Nexus. Nothing is wasted. Every concept you learn immediately improves your project.

How to Use This Codex

The `examples/` directory contains runnable code, tests, and Docker Compose setups for each stage. The `infra/` directory has Terraform modules for cloud deployment. After reading each stage, open the corresponding lab in `docs/` for hands-on exercises. The `doc/adr/` folder contains Architecture Decision Record templates to document your own choices. See `docs/portfolio-rubric.md` to evaluate your project against professional standards.

STAGE 1: BACKEND FUNDAMENTALS

Building the Authentication Service

STAGE 1 OBJECTIVE: Build a production-grade authentication service that handles registration, login, tokens, OAuth, 2FA, rate limiting, and session management. Tech: Node.js/Go + PostgreSQL + Redis + JWT.

1.1 Understanding Backend Engineering

When you open Instagram, you see photos. But where do those photos live? They live on servers — computers that are not your phone, sitting in data centers around the world.

The **backend** is everything that happens between the user's device and the database. It is the invisible machinery that makes modern applications work.

The Fundamental Question of Backend Engineering: How do we accept requests from millions of users, process them correctly, store data durably, and return responses — all in milliseconds?

The Full Request Lifecycle:

1. User taps "Login" on their phone
2. Phone sends HTTPS request to a load balancer
3. Load balancer forwards to an application server
4. Server validates input
5. Server queries the database
6. Server generates a response
7. Response travels back through the network
8. Phone renders the response on screen

Total time: ~200-500ms for most requests

1.2 REST APIs — The Language of the Web

Before REST, APIs were chaos. REST created a **standard language** that every system can speak.

REST = Representational State Transfer. It treats everything as a **resource** that can be:

- Created (POST)
- Read (GET)
- Updated (PUT/PATCH)
- Deleted (DELETE)

```
POST    /api/v1/auth/register    → Create account
POST    /api/v1/auth/login     → Authenticate
POST    /api/v1/auth/refresh   → Refresh tokens
POST    /api/v1/auth/logout    → Invalidate session
GET     /api/v1/auth/verify     → Verify email
POST    /api/v1/auth/forgot    → Password reset
POST    /api/v1/auth/2fa       → Verify 2FA
```

1.3 Authentication vs. Authorization

Authentication = Who are you? (Identity) **Authorization** = What can you do? (Permissions)

This is the single most misunderstood concept in backend engineering.

Authentication Flow (Complete):

1. User enters email & password
2. Client validates input format
3. Request hits load balancer → forwarded to auth service
4. Rate limit check (reject if > 5 attempts/min)
5. Query user by email from PostgreSQL
6. `bcrypt.compare(password, hash)`
7. Generate `access_token` (JWT, 15min expiry)
8. Generate `refresh_token` (opaque, 7 days)
9. Store refresh token hash in Redis (TTL: 7d)
10. Update `last_login` timestamp
11. Return { `access_token`, `refresh_token`, `user` }

1.4 Database Schema

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email       VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255),
  name        VARCHAR(255) NOT NULL,
  avatar_url  VARCHAR(500),
  role        VARCHAR(20) DEFAULT 'user',
  provider    VARCHAR(20) DEFAULT 'local',
  provider_id VARCHAR(255),

  email_verified BOOLEAN DEFAULT FALSE,
  verification_token VARCHAR(255),
  verified_at    TIMESTAMP,

  two_factor_enabled BOOLEAN DEFAULT FALSE,
  two_factor_secret  VARCHAR(255),
  two_factor_backup_codes TEXT[],

  failed_login_attempts INTEGER DEFAULT 0,
```

```

        locked_until          TIMESTAMP,
        last_login_at         TIMESTAMP,
        last_login_ip         INET,

        created_at            TIMESTAMP DEFAULT NOW(),
        updated_at            TIMESTAMP DEFAULT NOW()
    );

    CREATE INDEX idx_users_email ON users(email);
    CREATE INDEX idx_users_provider ON users(provider, provider_id);

```

1.5 JWT Implementation

```

interface TokenPayload {
    userId: string;
    email: string;
    role: string;
}

export function generateAccessToken(payload: TokenPayload): string {
    return jwt.sign(payload, process.env.JWT_ACCESS_SECRET!, {
        expiresIn: '15m',
        issuer: 'nexus-auth',
        audience: 'nexus-api'
    });
}

export function generateRefreshToken(userId: string): string {
    const token = crypto.randomBytes(40).toString('hex');
    return `${userId}.${token}`;
}

export function verifyAccessToken(token: string): TokenPayload {
    return jwt.verify(token, process.env.JWT_ACCESS_SECRET!) as TokenPayload;
}

```

1.6 Rate Limiting (Redis)

```

export async function rateLimit(req: Request, res: Response, next: NextFunction) {
    const key = `ratelimit:auth:${req.ip}`;
    const maxAttempts = 5;
    const windowMs = 60000;

    const current = await redis.incr(key);
    if (current === 1) await redis.expire(key, windowMs / 1000);

    if (current > maxAttempts) {
        const ttl = await redis.ttl(key);
        return res.status(429).json({

```

```

        error: 'Too many requests',
        retryAfter: ttl
    });
}

res.set('X-RateLimit-Limit', maxAttempts.toString());
res.set('X-RateLimit-Remaining', (maxAttempts - current).toString());
next();
}

```

1.7 Security: Password Storage

```

// ❌ WRONG — Never plain text, md5, or sha256
// ✅ CORRECT — bcrypt with cost factor 12
import bcrypt from 'bcrypt';

export async function hashPassword(password: string): Promise<string> {
    const salt = await bcrypt.genSalt(12);
    return bcrypt.hash(password, salt);
}

export async function verifyPassword(password: string, hash: string): Promise<boolean> {
    return bcrypt.compare(password, hash);
}

```

Why bcrypt? Slow by design (~250ms per attempt), salts every password uniquely, adaptive cost factor.

1.8 OAuth 2.0 Flow

OAuth solves **delegated authentication** — letting a third party verify identity without sharing passwords.

Flow:

1. User clicks "Login with Google"
2. App redirects to Google auth URL with client_id + redirect_uri + scope
3. User approves on Google's page
4. Google redirects back with authorization code
5. App exchanges code for access token + ID token
6. App verifies ID token (verify signature, check audience)
7. Find or create user by provider_id
8. Issue Nexus JWT + refresh token
9. User is logged in

1.9 Token Rotation Pattern

When a refresh token is used, ALWAYS rotate:

1. Parse and hash the incoming refresh token
2. Find matching token in database (unrevoked, not expired)
3. If not found — potential token theft. Revoke ALL tokens for user.
4. Revoke the old token
5. Issue new access + refresh token pair

Senior Engineer Insight: Token rotation isn't optional. Without it, a stolen refresh token is a permanent backdoor into your system.

Stage 1 Exercises

Reflection Questions:

1. Why separate access tokens (short-lived) from refresh tokens (long-lived)?
2. What happens if bcrypt cost factor is too low? Too high?
3. Why does rate limiting by IP alone not prevent distributed attacks?
4. What are the trade-offs of session-based vs. token-based auth?

Stage 1 Final Project Checklist:

- ☐ Registration with email verification
- ☐ Login with rate limiting
- ☐ JWT access tokens (15 min expiry)
- ☐ Refresh token rotation
- ☐ OAuth (Google + GitHub)
- ☐ Password reset flow
- ☐ 2FA with TOTP
- ☐ Redis session caching
- ☐ Account lockout after failed attempts
- ☐ Audit logging for all auth events

Stage 1 Lab

What to build: Production-grade auth service with JWT, OAuth, 2FA, rate limiting, and Redis sessions.

Minimum viable submission:

- Register, login, token refresh all working
- Unit tests passing (`npm test` in `examples/`)
- Docker Compose starts all services (`docker compose -f examples/docker-compose.dev.yml up`)

Resources: `docs/stage-1-testing.md` | `examples/tests/auth.unit.test.js` | `examples/auth/token.js` | `examples/auth/password.js`

STAGE 2: DATABASE ENGINEERING

From User Auth to Social Data

STAGE 2 OBJECTIVE: Design the database backends for Instagram, Spotify, Discord, and Netflix. Learn to model relationships, scale queries, and think like a DB engineer.

2.1 Why Databases Exist

In-memory data dies when the server restarts. Files work but don't support concurrent access, efficient querying, or atomicity. Databases solve: **durable, queryable, concurrent, consistent data storage**.

ACID:

- **Atomicity** — Transaction succeeds or fails completely
- **Consistency** — Data always satisfies constraints

- **Isolation** — Concurrent transactions don't interfere
- **Durability** — Committed data survives crashes

2.2 Instagram: The Follow Relationship

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  username VARCHAR(30) UNIQUE NOT NULL,  
  display_name VARCHAR(100),  
  bio TEXT,  
  avatar_url VARCHAR(500),  
  is_private BOOLEAN DEFAULT false,  
  follower_count INT DEFAULT 0,  
  following_count INT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE follows (  
  follower_id UUID REFERENCES users(id),  
  followee_id UUID REFERENCES users(id),  
  created_at TIMESTAMP DEFAULT NOW(),  
  PRIMARY KEY (follower_id, followee_id)  
);  
  
CREATE TABLE posts (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  caption TEXT,  
  image_url VARCHAR(500)[],  
  location VARCHAR(255),  
  like_count INT DEFAULT 0,  
  comment_count INT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE likes (  
  user_id UUID REFERENCES users(id),  
  post_id UUID REFERENCES posts(id),  
  created_at TIMESTAMP DEFAULT NOW(),  
  PRIMARY KEY (user_id, post_id)  
);  
  
CREATE TABLE comments (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  post_id UUID REFERENCES posts(id),  
  parent_id UUID REFERENCES comments(id),  
  content TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE INDEX idx_posts_user_created ON posts(user_id, created_at DESC);
CREATE INDEX idx_comments_post ON comments(post_id, created_at);
CREATE INDEX idx_likes_post ON likes(post_id);
```

2.3 How Feeds Work (Fan-Out)

Fan-Out on Write (Twitter-style):

- User posts → Write to followers' feed caches
- 100 followers = 100 writes
- Read: $O(1)$ — just read your feed from cache

Fan-Out on Read (Instagram-style):

- User posts → Write to own timeline only
- Read: Pull from all followed users, merge and rank
- Read: $O(n)$ — merge n feeds

Instagram's Hybrid Approach:

- Celebrities (>100K followers): Fan-out on read
- Regular users (<10K followers): Fan-out on write
- Everyone in between: Dynamic threshold

2.4 How Likes Scale

Wrong approach:

```
SELECT COUNT(*) FROM likes WHERE post_id = ?; -- Scans millions of rows
```

Right approach — Denormalized count:

```
UPDATE posts SET like_count = like_count + 1 WHERE id = ?;
```

This is **eventual consistency** — the count may be slightly off, but it's fast and good enough for display.

2.5 Discord: Chat Systems

The Pagination Problem: Discord can't use offset pagination (LIMIT 50 OFFSET 100000). It's too slow.

```
-- ❌ Slow — database scans all rows
SELECT * FROM messages WHERE channel_id = ? ORDER BY created_at DESC LIMIT 50 OFFSET 100000;

-- ✅ Fast — cursor-based pagination
SELECT * FROM messages WHERE channel_id = ? AND created_at < ? ORDER BY created_at DESC
LIMIT 50;
```

Key insight: Always paginate by value (cursor), not position (offset).

2.6 Indexes — The Key to Performance

Without index: Full table scan (slow with millions of rows) With index: $\log(n)$ lookup (fast with any size)

When to index: Columns used in WHERE, JOINS, ORDER BY, columns with high selectivity.

When NOT to index: Small tables (<1000 rows), columns rarely queried, columns with low selectivity (boolean).

Composite Indexes:

```
-- Query: WHERE status = 'active' AND created_at > '2024-01-01'
CREATE INDEX idx_orders_status_created ON orders(status, created_at);
-- Column order matters: Put high-selectivity columns first
```

2.7 The CAP Theorem

Database	Type	Consistency	Availability	Partition Tolerance
PostgreSQL	SQL	Strong	Good	Manual
MongoDB	NoSQL	Tunable	High	Automatic
Cassandra	NoSQL	Eventual	Very High	Automatic
Redis	Cache	Strong	High	Depends

Real-world rule: During a network partition, you must choose:

- **CP** (Consistency + Partition tolerance) — Bank transactions
- **AP** (Availability + Partition tolerance) — Social media feeds

2.8 Redis: Beyond Caching

```
// String – Session token
await redis.set(session:, token, 'EX', 3600);

// Set – Unique followers
await redis.sadd(ollowers:, followerId);

// Sorted Set – Feed timeline
await redis.zadd(eed:, timestamp, postId);
await redis.zrevrange(eed:, 0, 49);

// List – Recent notifications
await redis.lpush(
  otifications:, notification);
await redis.ltrim(
  otifications:, 0, 99);
```

Stage 2 Exercises

Reflection Questions:

1. Why does Instagram use different feed strategies for celebrities vs. regular users?
2. How would you design the database for a real-time chat app serving 10M users?
3. What are the trade-offs between normalized and denormalized data?

Stage 2 Final Project Checklist:

- ☐ Database schemas for users, posts, comments, likes
- ☐ Follow/unfollow system
- ☐ News feed generation
- ☐ Chat messaging tables
- ☐ Redis caching for hot data
- ☐ Proper indexes on all queries

Stage 2 Lab

What to build: Database schemas for Instagram-style social features — users, follows, posts, comments, likes, chats.

Minimum viable submission:

- ER diagram for social graph
- SQL schema with proper indexes and foreign keys
- Redis caching layer for feeds

Resources: docs/portfolio-rubric.md#stage-2-database-engineering

STAGE 3: CLOUD ENGINEERING (AWS)

Making Nexus Real

STAGE 3 OBJECTIVE: Deploy Nexus to AWS. Learn every major service by understanding WHY it exists. Tech: AWS (EC2, S3, RDS, Lambda, VPC, ELB, CloudFront, Route 53).

3.1 Why Cloud Computing?

Before cloud: Companies bought physical servers, waited weeks for delivery, paid for idle capacity. **After cloud:** Rent compute by the second, scale instantly, pay only for what you use.

IaaS (EC2): You manage OS, runtime, app. AWS manages servers, networking, power. **PaaS (Elastic Beanstalk):** You manage app code. AWS manages everything else. **SaaS (Gmail):** You just use it.

3.2 IAM — Identity and Access Management

Why IAM exists: To answer "Who can do what with which resources?"

Principle of Least Privilege: Every identity gets only the permissions it needs, nothing more.

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::nexus-production-assets/*"
}
```

Never use root credentials. Never use a single IAM user for everything. Always use roles for EC2 instances.

3.3 EC2 — Virtual Servers

Type	Family	Use Case	Example
t3	General purpose	Web servers, small DBs	t3.medium (2vCPU, 4GB)

c6g	Compute opt.	API servers	c6g.large (2vCPU, 4GB)
r6g	Memory opt.	Redis, caching	r6g.xlarge (4vCPU, 32GB)
p4d	GPU	ML training	p4d.24xlarge (96vCPU, 8 A100)

3.4 S3 — Object Storage

Why S3 exists: Store files — images, videos, backups — that don't fit in a database.

Storage Classes (cost per GB/month):

- Standard: .023
- Infrequent Access: .0125
- Glacier: .004 (archival)
- Glacier Deep Archive: .001

Presigned URLs for uploads:

```
const command = new PutObjectCommand({
  Bucket: 'nexus-uploads',
  Key: uploads//-,
  ContentType: contentType
});
return getSignedUrl(s3Client, command, { expiresIn: 3600 });
```

3.5 VPC — Virtual Private Cloud

Network architecture:

- **Public subnets:** Load balancers, NAT gateways, bastion hosts
- **Private subnets (app):** Application servers (no direct internet)
- **Private subnets (db):** RDS, ElastiCache (isolated)

Why? Defense in depth. If an app server is compromised, the attacker can't reach the database from the internet.

3.6 Load Balancers & Auto Scaling

How it works:

1. Traffic hits ALB (Application Load Balancer)
2. ALB terminates TLS, forwards to healthy targets
3. Auto Scaling Group monitors CPU/memory
4. CPU > 70% → Launch new instance and register with ALB
5. CPU < 30% → Deregister and terminate

3.7 Lambda — Serverless Compute

When to use Lambda: Async processing, image resizing, event-driven tasks, scheduled tasks. **When NOT to use Lambda:** Long-running requests (> 15 min), high-throughput APIs with consistent traffic.

3.8 RDS — Relational Database Service

Key decisions:

- **Multi-AZ:** Automatic failover to standby in another AZ
- **Read replicas:** Up to 15 replicas for read scaling
- **Provisioned IOPS:** Consistent performance for production
- **Automated backups:** 35-day retention

Stage 3 Exercises

Reflection Questions:

1. Why put databases in private subnets instead of public ones?
2. When should you use Lambda vs. EC2 vs. ECS?
3. What happens during an AZ failure with Multi-AZ RDS?

Stage 3 Final Project Checklist:

- ☐ VPC with public/private subnets across 3 AZs
- ☐ RDS PostgreSQL with Multi-AZ and read replicas
- ☐ S3 for user uploads with presigned URLs
- ☐ Lambda for async image processing
- ☐ ALB with HTTPS termination
- ☐ Auto Scaling group (min 2, max 20)
- ☐ CloudWatch dashboards and alarms
- ☐ Secrets Manager for all credentials
- ☐ CloudFront CDN for static assets

Stage 3 Lab

What to build: Deploy Nexus to AWS with VPC, RDS, S3, ALB, Auto Scaling, and CloudWatch monitoring.

Minimum viable submission:

- VPC with public/private subnets across 2 AZs
- RDS PostgreSQL accessible only from app tier
- S3 bucket with presigned URL upload
- ALB with health checks and Auto Scaling

Resources: `infra/environments/staging/` | `infra/modules/` | `infra/README.md`

STAGE 4: DOCKER

Containerizing the World

STAGE 4 OBJECTIVE: Containerize every service. Run the entire Nexus platform with one command: `docker compose up`. Tech: Docker, Docker Compose, Multi-stage builds, Container optimization.

4.1 Why Containers?

The problem: "It works on my machine!" — different OS, library versions, configurations.

The solution: Package the application with everything it needs to run.

VM vs Container:

- VMs virtualize hardware (each has guest OS)

- Containers virtualize the OS (share host kernel)

Result: Containers are smaller, faster to start, and more resource-efficient.

4.2 Multi-Stage Dockerfile

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

# Stage 2: Production
FROM node:20-alpine AS production
RUN addgroup -g 1001 -S nodejs
RUN adduser -S nexususer -u 1001
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER nexususer
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

Why multi-stage? Final image is 1/10 the size. No build tools, no source code, no secrets.

4.3 Docker Compose Architecture

```
version: '3.9'
services:
  gateway:
    build: ./gateway
    ports: ["443:443"]
    depends_on: [auth-service, api-service]
    networks: [nexus-net]

  auth-service:
    build: ./auth-service
    expose: ["3000"]
    environment:
      DATABASE_URL: postgresql://nexus:@postgres:5432/nexus
      REDIS_URL: redis://redis:6379
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    healthcheck:
      test: ["CMD", "wget", "--spider", "http://localhost:3000/health"]
    restart: unless-stopped
```

```

    networks: [nexus-net]

postgres:
  image: postgres:16-alpine
  expose: ["5432"]
  volumes: [postgres-data:/var/lib/postgresql/data]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U nexus"]
  networks: [nexus-net]

redis:
  image: redis:7-alpine
  expose: ["6379"]
  command: redis-server --appendonly yes
  volumes: [redis-data:/data]
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
  networks: [nexus-net]

volumes:
  postgres-data:
  redis-data:

networks:
  nexus-net:
    driver: bridge

```

4.4 Container Best Practices

Senior Engineer Rules:

1. **One process per container** — Don't run app and cron in the same container
2. **Stateless containers** — Never store data inside the container
3. **Immutable infrastructure** — Never SSH into a running container to fix things
4. **Specific tags** — Never use :latest in production
5. **Health checks** — Every container declares how to check it's alive
6. **Resource limits** — Every container specifies CPU/memory limits
7. **Non-root user** — Never run containers as root

Stage 4 Exercises

Reflection Questions:

1. Why is multi-stage build important for production images?
2. What's the difference between EXPOSE and ports in docker-compose?
3. Why should containers be stateless?

Stage 4 Final Project Checklist:

- ☐ Multi-stage Dockerfiles for every service
- ☐ docker-compose.yml for local development
- ☐ Health checks on every container
- ☐ Resource limits on every service

- ☐ Named volumes for persistent data
- ☐ Custom network for service discovery
- ☐ Non-root users in containers

Stage 4 Lab

What to build: Containerize all Nexus services with Docker Compose. Multi-stage builds, health checks, resource limits.

Minimum viable submission:

- Multi-stage Dockerfile for each service
- `docker-compose -f examples/docker-compose.dev.yml up` starts everything
- Health checks pass on all containers
- Non-root user in every container

Resources: `examples/Dockerfile` | `examples/docker-compose.dev.yml` | `examples/package.json`

STAGE 5: DISTRIBUTED SYSTEMS

From Monolith to Microservices

STAGE 5 OBJECTIVE: Split the monolith into independent services communicating via message queues. Tech: Microservices, Kafka/RabbitMQ, Circuit Breakers, Saga Pattern, API Gateway.

5.1 Why Distributed Systems?

Monolith problems: One deploy takes down everything, can't scale components independently, team coordination nightmare.

Microservice promise: Independent deployability, independent scaling, team autonomy, fault isolation.

5.2 Nexus Service Boundaries

Service	Owns	Tech
Auth Service	Authentication, tokens	Node.js + PG
User Service	Profiles, follows	Go + PG
Post Service	Posts, comments, likes	Node.js + PG
Notification Service	Push, email, in-app	Node.js + Redis
Search Service	Full-text search	Python + ES
AI Service	Embeddings, RAG	Python + PGVector
Payment Service	Billing, subscriptions	Java + PG

5.3 Inter-Service Communication

Synchronous (REST/gRPC): Service A calls Service B directly. Simple but creates coupling. Used for request-response patterns.

Asynchronous (Message Queue): Service A publishes event to queue. Service B consumes when ready. Used for:

- Notification sending (don't block the request)
- Search indexing (eventual consistency is fine)
- Analytics events (fire and forget)
- Cross-service workflows (Saga pattern)

5.4 RabbitMQ Exchange Types

- **Direct:** Routes by exact routing key (email → email-queue)
- **Topic:** Routes by pattern (user.* → user-events queue)
- **Fanout:** Broadcasts to all bound queues

5.5 Kafka: Log-Based Message System

Why Kafka over RabbitMQ?

- Higher throughput (millions of messages/second)
- Message replay (consumers can re-read from any offset)
- Log-based storage (durable by default with replication)
- Built-in partitioning for parallel processing

5.6 Circuit Breaker Pattern

```
class CircuitBreaker {
  state: 'CLOSED' | 'OPEN' | 'HALF_OPEN' = 'CLOSED';
  failureCount = 0;

  async call(fn) {
    if (this.state === 'OPEN') {
      if (enoughTimePassed()) this.state = 'HALF_OPEN';
      else throw new CircuitBreakerOpenError();
    }
    try {
      const result = await fn();
      this.state = 'CLOSED';
      this.failureCount = 0;
      return result;
    } catch (error) {
      this.failureCount++;
      if (this.failureCount >= 5) this.state = 'OPEN';
      throw error;
    }
  }
}
```

5.7 The Saga Pattern

In microservices, each service has its own database. Distributed transactions are impossible. **Solution: Saga** — a sequence of local transactions with compensating actions for rollback.

Example — Order Creation Saga:

1. Order Service: Create order (PENDING)
2. Inventory Service: Reserve items
3. Payment Service: Charge customer

4. Order Service: Update to CONFIRMED

If payment fails: Release inventory (compensating action), mark order FAILED.

5.8 Retry with Exponential Backoff

```
async function retryWithBackoff(fn, maxRetries = 3, baseDelay = 1000) {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try { return await fn(); }
    catch (error) {
      if (attempt === maxRetries) throw error;
      if (error.status >= 400 && error.status < 500) throw error;
      const delay = baseDelay * Math.pow(2, attempt - 1) * (0.5 + Math.random() * 0.5);
      await new Promise(r => setTimeout(r, delay));
    }
  }
}
```

Stage 5 Exercises

Reflection Questions:

1. When would you choose RabbitMQ over Kafka (and vice versa)?
2. Why is the Saga pattern necessary in microservices?
3. What happens if the API Gateway goes down?

Stage 5 Final Project Checklist:

- ☐ Auth Service, User Service, Post Service (independent)
- ☐ Notification Service (event-driven)
- ☐ Message queue for async events
- ☐ Circuit breakers on all service calls
- ☐ Retry with exponential backoff
- ☐ Saga pattern for multi-service operations
- ☐ API Gateway for routing
- ☐ Graceful degradation on failures

Stage 5 Lab

What to build: Split Nexus monolith into microservices. Add message queue, circuit breakers, and Saga pattern.

Minimum viable submission:

- Two independent services communicating via message queue
- Circuit breaker on at least one inter-service call
- Saga with compensating transaction for one flow
- API Gateway routing requests to correct service

Resources: [docs/portfolio-rubric.md#stage-5-distributed-systems](#)

STAGE 6: AI ENGINEERING

Building Intelligent Systems

STAGE 6 OBJECTIVE: Build a complete RAG (Retrieval Augmented Generation) pipeline. Not just calling APIs — understand every component. Tech: Embeddings, Vector DB, LLMs, Agents, RAG.

6.1 Beyond ChatGPT

Most AI products aren't one model — they're a **pipeline**: User Input → Preprocessing → Retrieval → Augmentation → Generation → Postprocessing → Response.

6.2 The RAG Pipeline

Ingestion Pipeline:

1. Document (PDF/HTML/Text)
2. Chunking (split into 512-token pieces with overlap)
3. Embedding generation (text-embedding-3-small → 1536-dim vectors)
4. Store in Vector DB + Search Index

Query Pipeline:

1. User query → query embedding
2. Vector search (cosine similarity)
3. Hybrid fusion with keyword search
4. Context assembly (top-K chunks)
5. LLM prompt construction
6. Streaming response

6.3 Document Chunking

```
class DocumentChunker {
  chunk(text, metadata) {
    // 1. Split by semantic boundaries (headings)
    // 2. Split large sections by paragraphs
    // 3. Split long paragraphs by token count (512 tokens)
    // 4. Include 50-token overlap for context continuity
    // 5. Attach metadata (documentId, page, heading)
  }
}
```

6.4 Vector Database (PGVector)

```
CREATE EXTENSION vector;

CREATE TABLE document_embeddings (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  chunk_id VARCHAR(255) NOT NULL,
  document_id VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  embedding vector(1536),
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
```

```
);

CREATE INDEX idx_embeddings_ivfflat
ON document_embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

6.5 Hybrid Search

```
SELECT id, content, metadata,
       1 - (embedding <=> query_embedding) as vector_score,
       ts_rank(content_tsv, plainto_tsquery('english', query)) as text_score,
       (0.3 * ts_rank(content_tsv, plainto_tsquery('english', query)) +
        0.7 * (1 - (embedding <=> query_embedding))) as hybrid_score
FROM document_embeddings
WHERE content_tsv @@ plainto_tsquery('english', query)
      OR 1 - (embedding <=> query_embedding) > 0.7
ORDER BY hybrid_score DESC
LIMIT 10;
```

6.6 Streaming LLM Responses

```
async function* streamChat(messages, context) {
  const completion = await openai.chat.completions.create({
    model: 'gpt-4-turbo',
    messages: [
      { role: 'system', content: buildPrompt(context) },
      ...messages
    ],
    stream: true,
    temperature: 0.3
  });
  for await (const chunk of completion) {
    const content = chunk.choices[0]?.delta?.content || '';
    if (content) yield content;
  }
}
```

6.7 AI Agents with Tool Calling

Agents can call tools: search documentation, create tickets, send emails, query databases. The LLM decides which tool to call based on the user's intent.

Tool format:

```
{
  "name": "search_documents",
  "description": "Search Nexus documentation",
  "parameters": {
```

```
"query": { "type": "string" },
"limit": { "type": "number", "default": 5 }
}
```

6.8 Memory Systems

- **Working memory:** Current conversation (Redis, 1 hour TTL, last 50 messages)
- **Short-term memory:** Recent sessions (7 days)
- **Long-term memory:** User preferences, learned patterns (Vector DB)
- **Episodic memory:** Past interactions with outcomes

Stage 6 Exercises

Reflection Questions:

1. Why is chunking strategy so important for RAG quality?
2. What are the trade-offs between embedding models?
3. When should you use an agent vs. a simple LLM call?

Stage 6 Final Project Checklist:

- ☐ Document ingestion pipeline (PDF, HTML, Markdown)
- ☐ Intelligent chunking with overlap
- ☐ Embedding generation
- ☐ Vector database with PGVector
- ☐ Hybrid search (vector + full-text)
- ☐ Streaming LLM responses
- ☐ RAG for question answering
- ☐ AI agents with tool calling
- ☐ Memory system (working + long-term)

Stage 6 Lab

What to build: RAG pipeline — ingest documents, chunk, embed, store in vector DB, retrieve, and answer with LLM.

Minimum viable submission:

- Document chunking with overlap
- Embedding generation (OpenAI or local model)
- Vector DB with similarity search
- RAG query that returns contextual answers

Resources: [docs/portfolio-rubric.md#stage-6-ai](#)

STAGE 7: SEARCH ENGINEERING

Beyond SQL LIKE Queries

STAGE 7 OBJECTIVE: Build a professional search engine with full-text, semantic, and hybrid search. Tech: Elasticsearch, OpenSearch, Ranking Algorithms.

7.1 Why Dedicated Search?

SQL's LIKE '%query%' does a full table scan. Can't rank results. Doesn't understand language.

A search engine uses an **inverted index**: word → list of documents containing it. This enables instant full-text search across millions of documents.

7.2 Elasticsearch Architecture

An Elasticsearch cluster has nodes with primary and replica shards. Documents are indexed across shards. Search queries are broadcast to all shards and results are merged.

Key concepts: Index → Type → Document. Sharding for scale, replication for durability.

7.3 Search Quality: BM25 Ranking

BM25 is the default ranking algorithm in modern search engines:

- **TF (Term Frequency):** How often the query term appears in the document
- **IDF (Inverse Document Frequency):** How rare the term is across all documents
- **Length normalization:** Shorter documents get a boost

"Kubernetes" appears in few docs → IDF = high → high score "The" appears in every doc → IDF = low → low score

7.4 Hybrid Search (Vector + Keyword)

For best results, combine:

1. **Keyword search (BM25):** Exact term matching, handles rare terms well
2. **Vector search (KNN):** Semantic similarity, handles synonyms well
3. **Fusion:** Weighted combination (e.g., 30% keyword + 70% vector)

7.5 Search Features

- **Autocomplete:** Edge n-gram tokenizer for prefix matching
- **Faceted search:** Filter by category, author, date range, tags
- **Highlighting:** Show matching terms in context
- **Fuzzy matching:** Handle typos with Levenshtein distance
- **Personalized ranking:** Boost results based on user history

Stage 7 Final Project Checklist:

- ☐ Elasticsearch cluster with replicas
- ☐ Custom analyzers with autocomplete
- ☐ Full-text search with BM25 ranking
- ☐ Vector/KNN search for semantic matching
- ☐ Hybrid search fusion
- ☐ Faceted search (by author, tag, date)
- ☐ Autocomplete/suggest
- ☐ Highlighted results
- ☐ Search analytics dashboard

Stage 7 Lab

What to build: Elasticsearch cluster with full-text, vector, and hybrid search. Autocomplete and faceted filtering.

Minimum viable submission:

- Elasticsearch index with custom analyzers
- Full-text search returning ranked results
- At least one aggregation (faceted filter)
- Autocomplete/suggest endpoint

Resources: docs/portfolio-rubric.md#stage-7-search

STAGE 8: OBSERVABILITY

Seeing What Your System Is Doing

STAGE 8 OBJECTIVE: Make every part of Nexus observable. Detect failures before users do. Tech: Logging, Metrics, Tracing, Alerting, SLOs.

8.1 The Three Pillars

1. **Logs** — What happened? (Debugging)
2. **Metrics** — What's the trend? (Alerting)
3. **Traces** — What's the path? (Performance analysis)

8.2 Structured Logging

Always log as structured JSON, not plain text. Include: timestamp, service, level, event, requestId, userId, and error details.

```
{ "timestamp": "2024-01-15T10:30:00Z", "service": "auth-service", "level": "ERROR", "event": "login.failed", "requestId": "abc-123", "userId": "user_42", "error": { "message": "Invalid password", "name": "AuthError" } }
```

8.3 Key Metrics: RED Method

Resource	Rate	Errors	Duration
API Gateway	requests/sec	5xx count	p50/p95/p99
Auth Service	logins/sec	failures	login time
Database	queries/sec	deadlocks	query time
Queue	messages/sec	failures	processing time

8.4 Distributed Tracing

Use OpenTelemetry to trace requests across services. Each request gets a trace_id that propagates through all service calls. This lets you see the full path of a request.

8.5 SLOs, SLIs, Error Budgets

- **SLI:** What we measure (latency p99 < 500ms)
- **SLO:** What we promise (99.9% of requests under 500ms)
- **Error budget:** 100% - SLO = allowed failures (0.1% = ~8.76 hours/year)
- **When budget is low:** Freeze deploys until error rate drops

8.6 Alerting

Severity Levels:

- P0 (<15 min): Service down, data loss
- P1 (<1 hr): Major feature broken
- P2 (<4 hr): Partial degradation
- P3 (<1 day): Non-critical issue
- P4 (best effort): Minor cosmetic issue

Stage 8 Final Project Checklist:

- ☐ Structured JSON logging across all services
- ☐ Prometheus metrics (RED method)
- ☐ Grafana dashboards for every service
- ☐ Distributed tracing with OpenTelemetry
- ☐ SLO definitions for critical paths
- ☐ Alerting rules (P0-P4)
- ☐ On-call runbooks
- ☐ Health check endpoints (/health, /ready)

Stage 8 Lab

What to build: Instrument Nexus with structured logs, Prometheus metrics, distributed tracing, and Grafana dashboards.

Minimum viable submission:

- Structured JSON logging in all services
- /metrics endpoint with Prometheus metrics
- At least one Grafana dashboard with latency + error panels
- SLO definition for auth service

Resources: `docs/observability-lab.md` | `infra/monitoring/prometheus-rules.yaml` | `docs/grafana-dashboard-guidance.md`

STAGE 9: KUBERNETES

Operating at Scale

STAGE 9 OBJECTIVE: Run Nexus on Kubernetes. Auto-scale, self-heal, rolling updates. From 1 to 100+ containers. Tech: Pods, Deployments, Services, Ingress, HPA.

9.1 Why Kubernetes?

Docker Compose problems at scale: No auto-healing, no auto-scaling, no rolling updates, no service discovery, no secret management.

Kubernetes solves: Run containers like a Google data center — automated deployment, scaling, and management.

9.2 Core Resources

Pod: The smallest deployable unit. One or more containers with shared network and storage.

Deployment: Declarative updates for Pods. Supports replicas, rolling updates, rollbacks, self-healing.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: nexus
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: nexus
      service: auth
  template:
    spec:
      containers:
        - name: auth-service
          image: nexusapp/auth-service:1.2.3
          ports:
            - containerPort: 3000
          livenessProbe:
            httpGet: { path: /health, port: 3000 }
          readinessProbe:
            httpGet: { path: /ready, port: 3000 }

```

Service: Stable network endpoint for a set of Pods. Load balances traffic.

```

apiVersion: v1
kind: Service
metadata:
  name: auth-service
  namespace: nexus
spec:
  selector:
    app: nexus
    service: auth
  ports:
    - port: 3000
      targetPort: 3000
  type: ClusterIP

```

Ingress: External HTTP/HTTPS routing with host/path-based routing, TLS termination, and rate limiting.

9.3 Horizontal Pod Autoscaler

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

```



```
metadata:
  name: auth-service-hpa
  namespace: nexus
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: auth-service
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

9.4 Kubernetes Production Best Practices

1. PodDisruptionBudget: Keep minimum pods available during maintenance
2. Network Policies: Micro-segmentation between services
3. Resource Quotas: Prevent one team from consuming all cluster resources
4. Readiness + Liveness probes: Dont send traffic to unhealthy pods
5. PreStop hooks: Graceful shutdown (drain connections before terminating)
6. Resource requests/limits: Every container must specify CPU and memory

Stage 9 Final Project Checklist:

- All services deployed as Deployments
- Services for internal communication
- Ingress with TLS termination
- ConfigMaps and Secrets
- HPA for auto-scaling (3-20 pods)
- PodDisruptionBudget (min 2 available)
- Liveness + Readiness probes
- Rolling update strategy
- Network policies for security
- Resource quotas per namespace

Stage 9 Lab

What to build: Deploy Nexus on Kubernetes with Deployments, Services, Ingress, HPA, and rolling updates.

Minimum viable submission:

- All services as Kubernetes Deployments with health probes
- Ingress with TLS
- HPA scaling on CPU (3-20 pods)
- Successful rolling update (zero-downtime deploy)

Resources: docs/portfolio-rubric.md#stage-9-kubernetes

STAGE 10: SOFTWARE ARCHITECTURE & SYSTEM DESIGN

Thinking Like an Architect

STAGE 10 OBJECTIVE: Design systems used by millions. Make principled trade-offs. Think at the architecture level. Systems: Instagram, Netflix, Uber, YouTube, Amazon, Discord, WhatsApp, ChatGPT.

10.1 The Architecture Mindset

Junior Engineer: Writes code, solves one problem. **Senior Engineer:** Designs systems, solves classes of problems. **Staff Engineer:** Defines strategy, solves organizational problems. **Principal/Architect:** Sets technical vision, influences industry.

10.2 The Architecture Decision Framework

Every architecture decision must answer:

1. **Functional requirements** — What must it do?
2. **Non-functional requirements** — How well must it do it? (latency, throughput, availability, durability)
3. **Constraints** — Time, budget, team size, tech stack
4. **Trade-offs** — No perfect solution exists
5. **Decision** — With clear rationale
6. **Impact** — On performance, cost, complexity, team

10.3 Instagram Architecture

Key challenges: 1B+ MAU, 100M+ photos/day, feed load < 500ms, 99.99% availability.

Architecture decisions:

- **Hybrid fan-out feed:** Celebrities (>100K followers) use fan-out on read; regular users use fan-out on write
- **Post storage:** S3 + CloudFront CDN for global delivery
- **Social graph:** Cassandra (high write volume, no joins needed)
- **Feed cache:** Redis sorted sets (sub-millisecond reads)
- **Photo upload:** Async via Kafka queue (dont block user)
- **ID generation:** Snowflake (globally unique, sortable, 64-bit)

10.4 Netflix Architecture

Key challenges: 200M+ subscribers, stream to any device, adaptive bitrate, global CDN.

Architecture decisions:

- **Open Connect CDN:** Netflix appliances inside ISP networks for local content delivery
- **Adaptive Bitrate (BOLA):** Monitor buffer and bandwidth, select optimal quality
- **Chaos Engineering:** Chaos Monkey randomly kills services to test resilience
- **Microservices:** 500+ microservices, each independently deployable
- **Asynchronous event bus:** Kafka for all inter-service communication

10.5 Uber Architecture

Key challenges: Real-time matching of riders/drivers, geospatial queries, surge pricing.

Key components:

- **H3 Geospatial Index:** Hexagonal grid for fast nearby-driver queries

- **Matching Engine:** Custom optimization engine (C++) for price-time matching
- **Dispatch:** Ringpop for consistent hashing of riders to dispatchers
- **Surge Pricing:** ML model based on demand/supply ratio

10.6 WhatsApp Architecture

Key challenges: 2B+ users, end-to-end encryption, real-time delivery, no message loss, minimal server storage.

Architecture decisions:

- **End-to-end encryption:** Signal Protocol — server never sees message content
- **Persistent connections:** Long-lived TCP connections per user (Ejabberd, Erlang-based)
- **Store-and-forward:** Messages stored on server (30 day TTL) if recipient offline
- **Minimal metadata:** Server stores only delivery status, not message content
- **Single-threaded per user:** All messages for a user processed sequentially

10.7 The System Design Interview Framework

1. Understand requirements (functional + non-functional)
2. Estimate scale (DAU, requests/sec, storage, bandwidth)
3. Design data model (entities, relationships, storage choice)
4. Design API (endpoints, request/response format)
5. Design high-level architecture (components and their interactions)
6. Deep dive components (database, caching, scaling, consistency)
7. Identify bottlenecks and trade-offs
8. Optimize and document decisions

Stage 10 Final Project Checklist:

- Architecture Decision Records (ADRs) for every major decision
- C4 diagrams (Context, Container, Component, Code)
- Data flow diagrams for critical paths
- Disaster recovery plan
- Cost analysis per service
- Security threat model
- Trade-off analysis document
- Evolution plan for next 12 months

Stage 10 Lab

What to build: Architecture Decision Records, C4 diagrams, disaster recovery plan, and trade-off analysis for Nexus.

Minimum viable submission:

- At least 3 ADRs covering major architectural decisions
- Context and Container diagrams (C4 model)
- Trade-off analysis comparing two architectural approaches
- Security threat model for auth service

Resources: [doc/adr/template.md](#) | [doc/adr/0001-record-architecture-decision.md](#) | [docs/portfolio-rubric.md#stage-10-architecture](#)

FINAL CHAPTER: THE 24-MONTH MASTERY ROADMAP

From Apprentice to Architect

This roadmap assumes 15-20 hours/week. Minimum time to Staff Engineer: 5-7 years. With focused effort: 2 years to strong senior level.

Months 1-3: Foundations

Topics: TypeScript, Go, REST APIs, Authentication, OAuth, JWT, PostgreSQL, Git, CI/CD basics.

Books: Clean Code (Martin), The Pragmatic Programmer (Hunt & Thomas), HTTP: The Definitive Guide.

Projects:

- Nexus Auth Service (complete Stage 1)
- Instagram DB schema design
- Deploy a personal portfolio with CI/CD

Skill level: Can build and deploy a production-quality API service with auth, database, and caching.

Months 4-6: Data & Scale

Topics: Advanced SQL, indexing, NoSQL (Cassandra/Mongo), Redis, AWS (EC2, S3, RDS, VPC, ELB), Docker, CAP Theorem.

Books: Designing Data-Intensive Applications (Kleppmann), Database Internals (Petrov).

Projects:

- Nexus database schemas for all services
- Deploy Nexus on AWS with VPC, ALB, Auto Scaling
- Containerize with Docker and docker-compose

Skill level: Can design scalable databases, deploy to cloud, and containerize applications.

Months 7-9: Distributed Systems

Topics: Microservices, message queues (Kafka/RabbitMQ), circuit breakers, Saga pattern, API Gateway, gRPC.

Books: Building Microservices (Newman), Designing Distributed Systems (Burns).

Projects:

- Split Nexus into microservices
- Implement event-driven communication
- Add circuit breakers and retry logic

Skill level: Can decompose monoliths, design async communication, handle distributed failures.

Months 10-12: AI & Search

Topics: Embeddings, vector databases, RAG pipelines, LLM APIs, AI agents, Elasticsearch, BM25 ranking, hybrid search.

Books: Designing Machine Learning Systems (Huyen), AI Engineering.

Projects:

- Add RAG pipeline to Nexus
- Implement hybrid search with Elasticsearch
- Build AI agents with tool calling

Skill level: Can build AI-powered features and professional search into any application.

Months 13-15: Production Engineering

Topics: Observability (logs, metrics, traces), Prometheus, Grafana, OpenTelemetry, SLOs, incident response.

Books: Site Reliability Engineering (Google), The Art of Monitoring (Turnbull).

Projects:

- Add structured logging, metrics, and tracing to Nexus
- Create Grafana dashboards
- Define SLOs and alerting rules
- Write incident runbooks

Skill level: Can operate production systems, detect failures before users, and run on-call.

Months 16-18: Kubernetes & Orchestration

Topics: Kubernetes (Pods, Deployments, Services, Ingress, HPA, ConfigMaps, Secrets), Helm, service mesh.

Books: Kubernetes in Action (Lukša), The Kubernetes Book (Poulton).

Projects:

- Deploy Nexus on Kubernetes
- Configure HPA, PDB, network policies
- Set up CI/CD pipeline for K8s

Skill level: Can operate Kubernetes clusters and deploy production workloads.

Months 19-21: System Design Mastery

Topics: Design Instagram, Netflix, Uber, WhatsApp, YouTube, Amazon, Stripe, ChatGPT. Trade-off analysis, capacity planning.

Books: System Design Interview (Xu), Designing Data-Intensive Applications (re-read).

Projects:

- Write architecture decision records for Nexus
- Create C4 diagrams for the full system
- Conduct mock system design interviews

Skill level: Can design systems serving 100M+ users and ace system design interviews.

Months 22-24: Integration & Leadership

Topics: Technical strategy, architecture reviews, mentoring, cross-team communication, production excellence.

Books: The Staff Engineer's Path (Reilly), An Elegant Puzzle (Larson).

Projects:

- Mentor junior engineers on Nexus codebase
- Write technical strategy document for Nexus v2
- Contribute to open source

Skill level: Can define technical strategy, mentor engineers, and influence architecture decisions org-wide.

The Continuous Learning Path

Beyond 24 months, rotate through these areas:

- **Performance engineering:** Profiling, optimization, benchmarking
- **Security engineering:** Threat modeling, penetration testing, security architecture
- **Developer experience:** Platforms, internal tools, productivity
- **Research:** Read papers, attend conferences, contribute to the community

Final Words

Architecture is not about knowing all the answers. It is about asking the right questions.

Every system you design will have trade-offs. The goal is not to build the perfect system — that doesn't exist. The goal is to make intentional, well-understood trade-offs that align with your requirements.

You will make mistakes. Everyone does. The best engineers learn from production incidents. The best architects question their own decisions.

The fact that you've made it to the end of this handbook means you have the curiosity and dedication required to become an exceptional engineer. Now go build something that matters.

Appendix: Repository Structure

```
.
├── THE-SOFTWARE-ARCHITECT-CODEX.md # This handbook
├── .github/workflows/ci.yml        # CI pipeline (GitHub Actions)
├── examples/                      # Runnable code, Docker Compose, tests
│   ├── auth/token.js              # JWT + refresh token implementation
│   ├── auth/password.js           # bcrypt password hashing
│   ├── tests/auth.unit.test.js    # Unit tests for auth
│   ├── docker-compose.dev.yml     # Local development environment
│   ├── Dockerfile                 # Multi-stage build
│   └── package.json
├── infra/                         # Infrastructure as Code (Terraform)
│   ├── README.md                  # How to bootstrap and apply
│   ├── modules/                   # Reusable Terraform modules
│   │   ├── network/              # VPC, subnets, security groups
│   │   ├── postgres/             # RDS Aurora PostgreSQL
│   │   └── redis/                 # ElastiCache Redis
│   ├── environments/staging/      # Staging environment config
│   └── monitoring/
│       └── prometheus-rules.yaml  # Alerting rules
├── docs/                          # Labs, guides, and rubrics
│   ├── stage-1-testing.md         # Testing chapter
│   ├── observability-lab.md       # Observability hands-on
│   ├── grafana-dashboard-guidance.md # Dashboard setup guide
│   └── portfolio-rubric.md        # Evaluation criteria
└── doc/adr/                       # Architecture Decision Records
    ├── template.md               # ADR template
    └── 0001-record-architecture-decision.md # Sample ADR
```

Remember: Knowledge is knowing what to build. Wisdom is knowing what NOT to build.
